

College of Design
and Engineering

Course	☐ EG3611A (IA)	☐ EG3612 (VIA)
Report	☐ Interim	☐ Final

Reporting Period	**11 May 2026**	to	**31 July 2026**

Name	Karthikeyan Vetrivel
Matriculation No.	A0307459W
Engineering Discipline	Computer Engineering
Internship Period	11 May 2026 to 31 July 2026
Mentor	Ramesh Iyer Srikrishna
Company	ST Engineering

Table of Contents

ACKNOWLEDGEMENTS ..... **ii**
INTRODUCTION **1**
PROJECT DETAILS **1**
Industry and Employer Overview 1
Project Background and Objectives 2
Work Performed and Objectives Accomplished 3
Research Reconstruction and Development Environment 3
Stage 1: Privileged Expert 4
Stage 2: Depth-Student Distillation 6
Stage 3: PPO Fine-Tuning 7
Evaluation 8
Challenges 8
Reflections 9
Possible Career Paths 9
CONCLUSION **10**
REFERENCES **11**
APPENDIX **12**
WEEKLY JOURNAL .....

ACKNOWLEDGEMENTS

I would like to thank my supervisor at AI.R STC, ST Engineering, for guiding me throughout the internship. He shared useful learning resources, patiently clarified concepts that I was uncertain about and gave me the freedom to explore different approaches to improve my learning. I am also grateful to National University of Singapore (NUS) for giving me the opportunity to undertake this credit-bearing internship.

INTRODUCTION

From 11 May to 31 July 2026, I completed a three-month internship as an AI & Robotics Intern at AI.R STC, ST Engineering. During the internship, I had the opportunity to work on a project that explored how reinforcement learning could be used to train the Unitree B2W, a wheeled quadruped robot, to navigate stairs, slopes, debris and other uneven terrain. The project was motivated by urban search-and-rescue scenarios, where a robot may need to enter a hazardous environment while maintaining stable and reliable locomotion.

My work involved reconstructing and adapting the training framework presented in *Parkour in the Wild* (Rudin et al., 2025). The framework had three main stages: training terrain-specialised expert policies, distilling their behaviours into a deployable student policy that used depth-camera observations, and improving the student through reinforcement-learning fine-tuning. In this report, I will explain how I developed the project, the challenges I encountered, how I responded to them, and the lessons I will apply to my future studies and career.

PROJECT DETAILS

Industry and Employer Overview

The robotics industry can broadly be divided into industrial robots used in manufacturing and service robots used in areas such as logistics, healthcare, security and emergency response. Physical AI refers to intelligent machines that can perceive their surroundings, make decisions and act in the physical world. Unlike software-only systems, these machines must handle sensor uncertainty, real-time control requirements, safety constraints and the gap between simulation and reality.

In 2024, nearly 200,000 professional service robots were sold, representing a 9% increase, while sales of search-and-rescue and security robots rose by 19% to approximately 3,100 units (International Federation of Robotics, 2025). Key robotics trends for 2026 include AI-enabled autonomy, the integration of information technology and operational technology, improved safety and cybersecurity, and the use of robots to address labour shortages (International Federation of Robotics, 2026). These trends show a growing demand for reliable robots that can operate in unfamiliar environments.

ST Engineering is a global technology, defence and engineering group with three main business areas: Commercial Aerospace, Smart City & Digital Solutions, and Defence & Public Security (ST Engineering, n.d.). Physical AI is one of the company's strategic priorities. In September 2025, the company announced a five-year, S$250 million AI Research Translation programme and introduced the **Manned-Unmanned Teaming Operating System (MUMTOS)**. This AI-powered command-and-control platform coordinates robots, drones and autonomous vehicles working alongside human operators (ST Engineering, 2025). To support this direction, ST Engineering aims to develop 5,000 AI engineers by upskilling 4,000 existing engineers and creating 1,000 specialist roles in AI-module development, cybersecurity for AI and agentic AI systems (ST Engineering, 2025).

I was attached to the Group Technology Office (GTO), which works across the different Business Units within ST Engineering and provides research and development support where needed.



Figure 1. ST Engineering organisational structure

Project Background and Objectives

The project I worked on during this internship was titled **"RL-Based Training Framework for Robust Wheeled-Quadruped Locomotion."**

It focused on the **Unitree B2W**, which has twelve leg joints and four powered wheels. My role was to train policies that could eventually be deployed on the robot, with the aim of making its locomotion more robust and adaptable across different types of terrain.



Figure 2. Unitree B2W wheeled quadruped robot

The intended use case was urban search-and-rescue operations in hazardous environments containing debris, stairs and confined spaces. The Land Systems Business Unit was already using the Unitree B2W for autonomous patrol navigation. However, this work was developed at the application layer, while the lower-level locomotion controller still relied on the default factory policy. This meant that the robot could navigate towards a target point but might still fail, or behave unexpectedly, when it encountered out-of-

distribution terrain. My internship therefore focused on building a framework for training a more robust locomotion policy that could eventually be deployed on the B2W.

I worked on a teacher-student distillation framework in which skill-specific expert policies were trained and their behaviours were distilled into a single generalist student policy. I based the framework on *Parkour in the Wild* (Rudin et al., 2025). The method first trains specialist policies, distils them into a unified foundation policy through Dataset Aggregation (DAgger), and then fine-tunes that policy using reinforcement learning on a wider terrain distribution. As several details about expert training were not fully specified, I also referred to *ANYmal Parkour* and other supporting work (Hoeller et al., 2024; Ross et al., 2011; Rudin et al., 2022; Schulman et al., 2017). The following sections explain how I reconstructed and developed this framework during the internship.

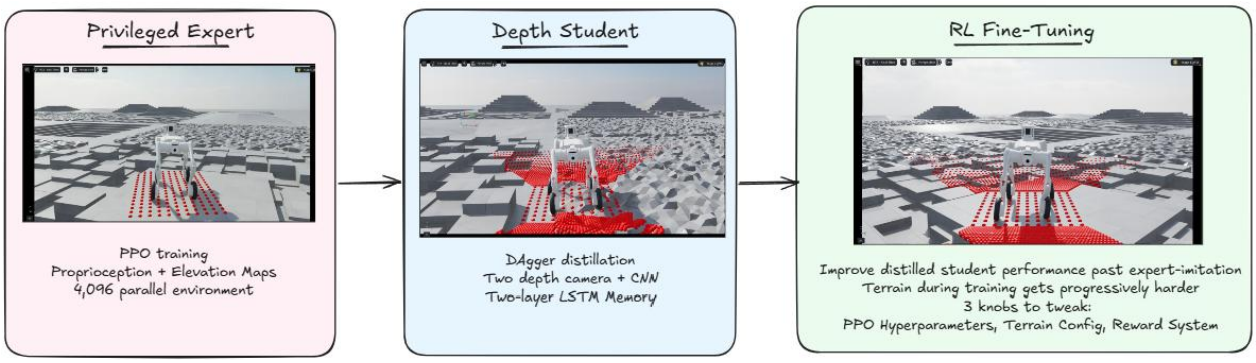


Figure 3. Unitree B2W training workflow

Work Performed and Objectives Accomplished

Research Reconstruction and Development Environment

At the start of the project, I had to build a stronger foundation in deep learning, reinforcement learning and policy optimisation. Although the learning curve was steep, I found the concepts interesting and gradually became more comfortable applying them. I also studied concepts such as DAgger-style imitation learning, Proximal Policy Optimisation (PPO) to improve my understanding.

A **policy** is the robot's decision-making model. It receives the desired command and the robot's current observations, then produces the actions that the robot should take. These actions change the environment and generate new observations, which are fed back into the policy. This process repeats continuously while the robot is moving.

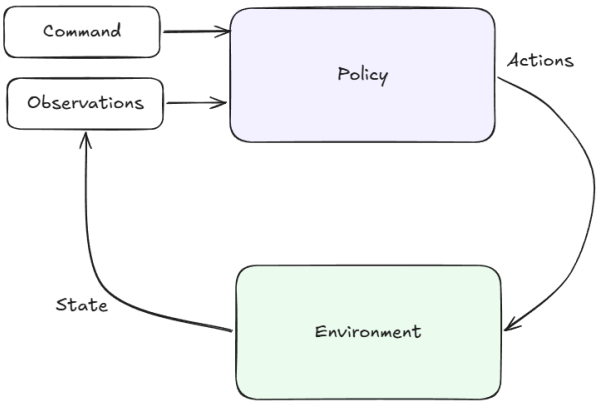


Figure 4. Policy-environment control loop

I used the following platforms during development:

Tool / Technology	Purpose
NVIDIA Isaac Sim 5.1	Provide physics simulation environment for training and testing
Isaac Lab 2.3.2	Support development of RL environments, terrain config and robot tasks
rsl_rl	RL training library
PyTorch	Used to develop, train and modify the neural-network models
Weights & Biases (WandB)	Used to track experiments, training metrics and model performance.
GitHub	Version Control: near-locomotion-quadruped

Table 1. Platforms used for development

I consolidated the workflow in a public project repository that included setup instructions, training and evaluation commands, terrain-generation notes, checkpoint handling and deployment preparation. I also documented the pipeline carefully so that it could be reproduced and handed over clearly at the end of my internship.

Stage 1: Privileged Expert

In the first stage of the pipeline, I trained the expert policy using reinforcement learning with Proximal Policy Optimisation (PPO).

Proximal Policy Optimisation (PPO) was the reinforcement-learning algorithm used to train the policy. It collected observations, actions and rewards from the robot's interactions with the environment, then updated the policy to favour actions that produced better outcomes. At the same time, it limited how much the policy could change during each update, which helped to keep training stable. For this project, I used Isaac Sim to collect rollouts from 4,096 environments in parallel and calculate the training loss from a much larger batch of experience.

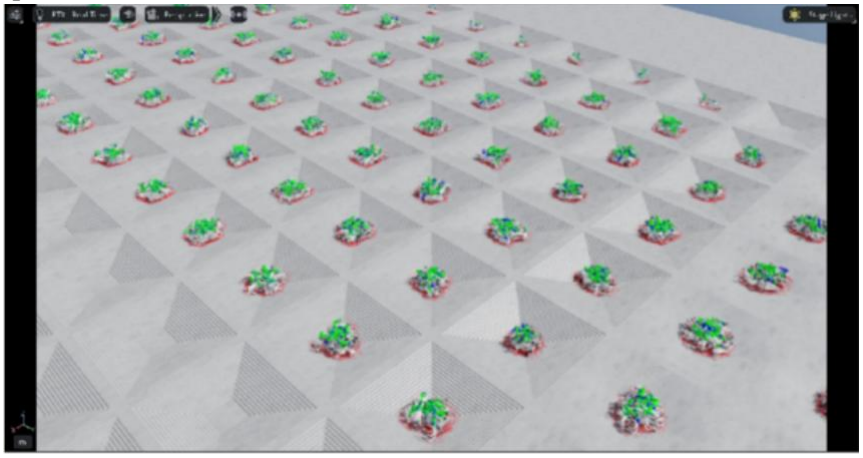


Figure 5. 4,096 parallel training environments in Isaac Sim

The expert policy used the following observations:

Observation	Dimension / description
Base Linear Velocity (base_lin_vel)	3 dim (vx, vy, vz in base frame)
Angular Velocity (base_ang_vel)	3 dim (wx, wy, wz in base frame)
Gravity Vector (projected_gravity)	3 dim (unit gravity in base frame)
Joint Positions (joint_pos)	16 dim
Joint Velocities (joint_vel)	16 dim
Last actions (actions)	16 dim
1.6 m × 1.0 m elevation map (height_scan)	187 dim
Velocity Command (velocity_commands)	3 dim (lin_x, lin_y, ang_z)

Table 2. Expert policy observation dimensions

The expert policy had a total input of **247 dimensions**.

The privileged walking expert used 247 observations in total, consisting of 60 proprioceptive and command observations and a 187-point elevation map. The elevation map gave the policy information about the terrain around the robot, allowing it to anticipate stairs, slopes and obstacles before reaching them. In comparison, an expert without elevation-map observations relied mainly on the robot's internal state and could only react after its body or wheels encountered a change in terrain.

I also included the base linear velocity to give information on how it was actually moving along the x, y and z axes. This allowed the policy to compare its current velocity with the commanded velocity and correct any tracking errors. It also helped the expert detect slipping, slowing down or unintended vertical movement while travelling over uneven terrain.

Result: Due to the time available, I was only able to fully train the walking expert. Adding the elevation-map observations and base linear velocity increased the **episode return (Rewards)** from approximately **273 to 287**, while reducing the **velocity-tracking error** from **0.48 m/s to 0.25 m/s**. This suggested that the policy had learnt a more effective gait that followed the commanded velocity and handled uneven terrain more successfully. The training metrics also converged, indicating that the expert had learnt stable locomotion behaviour within the simulated training environment.

The same framework can be extended to train other expert policies for skills such as climbing stairs, travelling on slopes, crossing high obstacles and overcoming gaps. Each expert would require a reward system and terrain configuration designed for that specific skill.

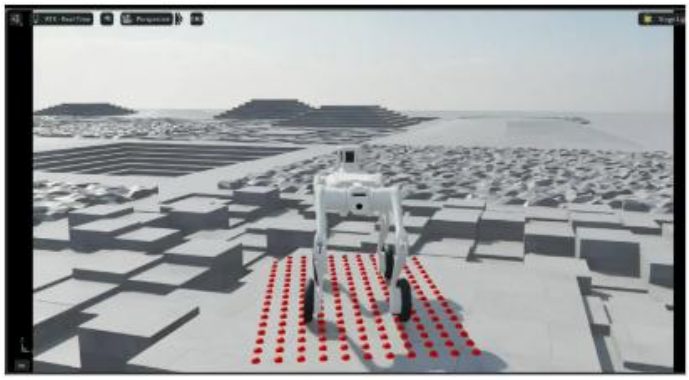


Figure 6. Walking expert using elevation maps in out-of-distribution terrain

Stage 2: Depth-Student Distillation

Stage 1 is designed to produce multiple experts, with each expert specialising in a particular skill. To allow the robot to handle different obstacles without manually switching between these experts, their behaviours need to be distilled into a single policy.

For multi-expert distillation, the training terrain would combine the terrain configurations used for the individual experts so that the student policy could learn from them together. As there was not enough time to train multiple experts during my internship, I implemented the multi-expert framework but used only the walking expert for the student-distillation run.

I used DAgger to perform the distillation. During training, the student acted independently while the expert continuously provided the correct action for each state that the student encountered. This allowed the student to learn from the situations created by its own behaviour.

Since the physical B2W cannot directly obtain elevation maps, the deployable student replaced the elevation scan with two simulated 48 × 32 depth images. It also retained 57 proprioceptive values and a three-dimensional velocity command. Each camera used an independent CNN encoder. The camera features and proprioceptive observations were then passed into a two-layer LSTM with a hidden size of 256. Its output was combined with a proprioceptive skip connection and the command before entering the policy head. I developed this custom CNN-LSTM arrangement because the rsl_rl library provided convolutional and recurrent modules separately, but not in the combined form required for this project.

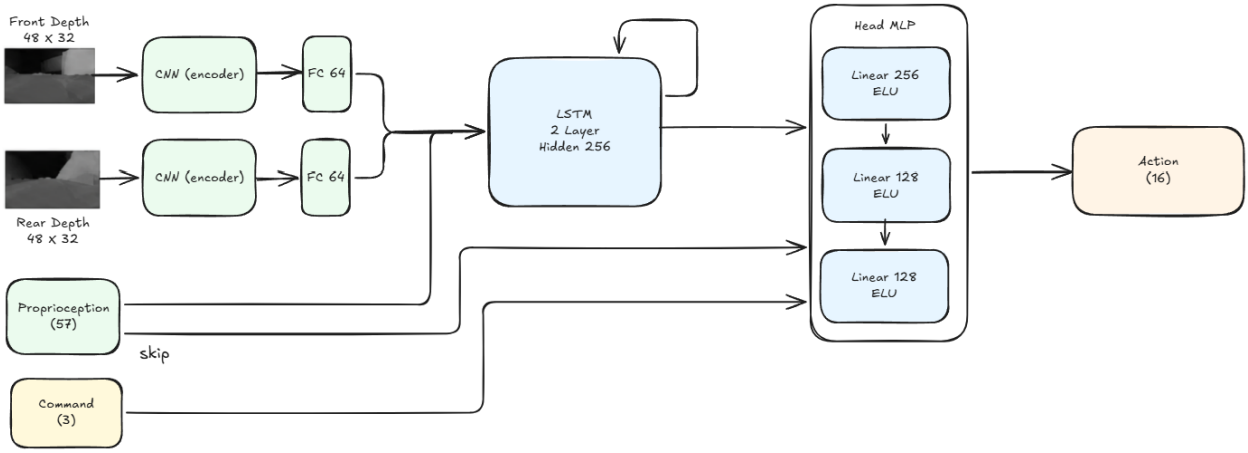


Figure 7. Student policy neural-network architecture

Following Rudin et al. (2025), I degraded the simulated depth images to better resemble real camera data.

This involved clipping unrealistic depth values, adding noise around object edges, creating missing patches that remained consistent over time using Perlin noise (Perlin, 2002), removing one to five columns from the left side to simulate a camera blind spot, and applying Gaussian blur. This was necessary to close the Sim2Real (Simulation to Reality) Gap.

The cameras updated at 15 Hz while the policy ran at 50 Hz. During DAgger-style distillation, the student visited states under its own policy while the frozen expert provided the target actions.

Result: The **behaviour loss** fell from **25.9 to approximately 0.153**, showing that the recurrent depth student had learnt to approximate the privileged walking teacher.

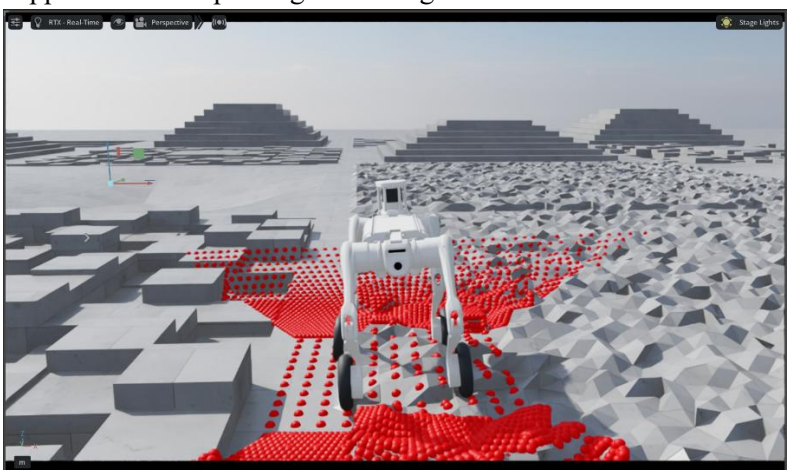


Figure 8. Distilled student traversing out-of-distribution terrain

Stage 3: PPO Fine-Tuning

After multi-expert distillation, the student policy needed to be further optimised using reinforcement learning. During distillation, the student only learnt to imitate the actions produced by the expert. However, Rudin et al. (2025) reported that performance on individual skills could decrease and that generalisation to unseen terrain could remain limited.

Rudin et al. (2025) identified three requirements for effective fine-tuning: preserving the foundation policy's **performance and robustness** under action noise, tuning the **RL hyperparameters** conservatively, and efficiently **pre-training the critic network**.

Since the distillation stage trained only the actor, I first had to pre-train the critic before fine-tuning the actor. I continued using PPO, but kept the actor frozen so that only the critic was updated. After the critic had stabilised, I unfroze the actor and continued fine-tuning.

Phase A: I loaded only the actor from student distillation, kept it frozen and trained a randomly initialised critic.

Phase B: After the critic had stabilised and could provide a more accurate learning signal, I loaded it from Phase A, unfroze the actor and continued training.

The main settings I needed to tune were:

1. PPO hyperparameters
2. Reward system that supports all experts
3. Terrain configuration combining all expert terrains

As I was using only one expert, I did not need to adjust the reward system or terrain configuration for this

run. My main focus was tuning the PPO hyperparameters.

Result: During Phase A, the model converged to a **mean reward** of **290.5** and a **critic loss** of **0.136** while the actor remained frozen.

After I unfroze the actor in Phase B, the **reward** initially peaked near **299 before gradually falling to 269.9**, showing that the training had **not converged**. Although the **velocity-tracking error** improved slightly from **0.342 m/s to 0.318 m/s**, the **body-rocking penalty** worsened from **−0.0339 to −0.165**. This suggested greater body rocking, which could lead to unnecessary power use.

The policy therefore needs to be retrained. Instead of continuing through trial and error, I documented the PPO hyperparameters and reward system so that future changes could be tested with a clearer hypothesis. However, due to the limited time available, I was unable to complete enough runs to determine which hyperparameter combination worked best.

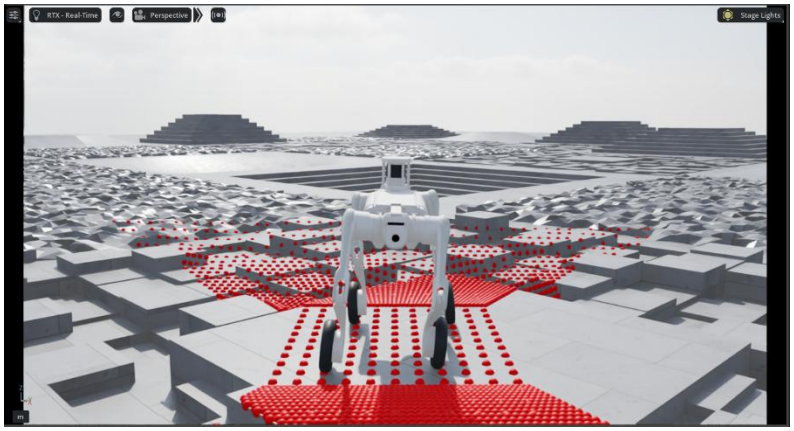


Figure 9. RL-fine-tuned student traversing out-of-distribution terrain

Evaluation

To evaluate the policies consistently, I implemented an evaluation script based on the method used in Table 4 of *Parkour in the Wild* (Rudin et al., 2025). Each policy was tested using 1,000 simulated robots on terrain set to 90% of the training difficulty, and a trial was recorded as successful when the robot travelled the required distance without falling.

Policy	Expert success	Student success	Fine-tuned student success
My Policy	93.3%	82.8%	96.1%
Parkour in the Wild	94.60%	99.30%	100%

Table 3. Simulation success rates at 90% terrain difficulty

Although the percentages are not directly comparable because the robots and evaluation conditions differed, both results show that the fine-tuned student achieved the highest success rate. My distilled student initially performed below the expert (82.8% versus 93.3%) but improved to 96.1% after fine-tuning, whereas the paper's student already exceeded its expert (99.3% versus 94.6%) and reached 100% after fine-tuning.

Challenges

Reconstructing a Distributed Research Pipeline. The first challenge was reconstructing the three-stage framework presented in *Parkour in the Wild* (Rudin et al., 2025). There was no complete source code

available, and important details about expert training, reward design, DAgger distillation and PPO were spread across several papers developed for the ANYmal D. To address this, I cross-referenced the supporting research, adapted the methods for the Unitree B2W and documented the observations, rewards, configurations and implementation assumptions. This gave me a clearer and more reproducible training framework.

RL Fine-Tuning of the Distilled Student. The second challenge was fine-tuning the distilled student using PPO without overwriting the behaviour it had already learnt. I studied PPO using OpenAI's Spinning Up documentation (OpenAI, n.d.) and connected the theory to the project implementation. During the run, the reward initially peaked near 299 before declining to 269.9, while the body-rocking penalty worsened from −0.0339 to −0.165 because excessive exploration degraded the policy. Although I was unable to complete the fine-tuning within the internship, the experience taught me that RL tuning should be guided by a clear hypothesis and evaluated using reward, entropy, tracking and stability together.

Reflections

The internship was initially challenging because the project required knowledge across reinforcement learning, robotics, perception and simulation. However, my supervisor supported me by sharing relevant resources, patiently explaining concepts that I was uncertain about and setting clear expectations for my deliverables. More importantly, he gave me the freedom to explore different approaches and learn through experimentation. This balance between guidance and independence helped me become more confident, resourceful and willing to take ownership of my work. What also stood out to me was how welcoming and open everyone at the branch was. Colleagues from different departments were willing to share their knowledge and discuss the projects they were working on. The academic huddles conducted by graduates and PhD students introduced me to research beyond the scope of my own project and showed me how technical ideas could be communicated effectively. Overall, the internship allowed me to develop both technically and professionally, while strengthening my interest in Physical AI and encouraging me to pursue further opportunities in robotics and artificial intelligence.

Possible Career Paths

The internship strengthened my interest in Physical AI and helped me identify several career paths that relate to my experience. Robotics AI and Machine Learning Engineers develop perception and control systems for autonomous machines, while Reinforcement Learning Engineers focus more specifically on learning algorithms, reward design and policy evaluation. I am also interested in robot perception because the student policy relied on depth-image encoding and temporal memory to understand its surroundings. Embedded AI and robotics deployment roles would allow me to extend this work through model optimisation, ROS 2 controller integration and the validation of real-time behaviour on physical hardware. Another area that interests me is research translation, where academic advances are adapted for practical robotic applications. Through future coursework, research attachments and projects in robotics, embodied intelligence and reinforcement learning, I hope to determine which career path best aligns with my interests and strengths.

CONCLUSION

Through this internship, I was able to see how learning-based robotics research could be translated into an engineering workflow from start to finish. I reconstructed the core stages of *Parkour in the Wild* (Rudin et al., 2025) for the Unitree B2W, trained a privileged walking expert, developed a CNN-LSTM depth student, implemented preliminary PPO fine-tuning and created a repeatable evaluation process. The selected fine-tuned checkpoint met the in-domain target. However, OOD robustness was still limited, and I was not able to complete multi-expert consolidation, depth-student Sim2Sim integration or hardware deployment within the internship.

Beyond the technical results, I became more comfortable learning from research papers, asking focused questions, documenting my decisions and taking responsibility for work that involved uncertainty. The guidance and independence provided by my supervisor, together with the branch's open culture, strengthened my interest in Physical AI. These experiences also prepared me to approach future robotics projects in a more structured way, especially when the work involves uncertainty and safety.

REFERENCES

Hoeller, D., Rudin, N., Sako, D., & Hutter, M. (2024). ANYmal parkour: Learning agile navigation for quadrupedal robots. *Science Robotics, 9*(88), eadi7566. https://doi.org/10.1126/scirobotics.adi7566

International Federation of Robotics. (2025, October 7). *Service robots see global growth boom*. https://ifr.org/ifr-press-releases/news/service-robots-see-global-growth-boom

International Federation of Robotics. (2026, January 8). *Top 5 global robotics trends 2026*. https://ifr.org/ifr-press-releases/news/top-5-global-robotics-trends-2026

OpenAI. (n.d.). *Proximal policy optimization*. Spinning Up in Deep RL. https://spinningup.openai.com/en/latest/algorithms/ppo.html

Perlin, K. (2002). Improving noise. *ACM Transactions on Graphics, 21*(3), 681–682. https://doi.org/10.1145/566654.566636

Ross, S., Gordon, G., & Bagnell, D. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. In G. Gordon, D. Dunson, & M. Dudík (Eds.), *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics* (Vol. 15, pp. 627–635). PMLR. https://proceedings.mlr.press/v15/ross11a.html

Rudin, N., He, J., Aurand, J., & Hutter, M. (2025). *Parkour in the wild: Learning a general and extensible agile locomotion policy using multi-expert distillation and RL fine-tuning* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2505.11164

Rudin, N., Hoeller, D., Reist, P., & Hutter, M. (2022). Learning to walk in minutes using massively parallel deep reinforcement learning. In A. Faust, D. Hsu, & G. Neumann (Eds.), *Proceedings of the 5th Conference on Robot Learning* (Vol. 164, pp. 91–100). PMLR. https://proceedings.mlr.press/v164/rudin22a.html

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal policy optimization algorithms* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1707.06347

ST Engineering. (n.d.). *Organisation structure*. Retrieved August 2, 2026, from https://www.stengg.com/en/about-us/organisation-structure/

ST Engineering. (2025, September 4). *ST Engineering spotlights AI innovation at 5th InnoTech conference*. https://www.stengg.com/en/newsroom/news-releases/st-engineering-spotlights-ai-innovation-at-5th-innotech-conference/

APPENDIX

Appendix A: Policy Architecture

Component	Expert policy	Student policy
Proprioceptive observations	57	57
Velocity command	3	3
Elevation map	187	–
Depth-camera input	–	Two 48 × 32 images
CNN output	–	64 per camera
LSTM input	–	185
LSTM hidden size	–	256
Policy-head input	–	316
Action output	16	16
Total expert input	247	–

Weekly Journal

The weekly journal should serve as your work log diary. Each entry should highlight one key learning point each week and include a short description. Bullet points are accepted.

You may record your thoughts on how your work tasks contribute to the performance or targets of your business unit. You may also identify perspectives that have changed as a result of your work experiences, including interactions with your managers and colleagues.

Attach this journal to your internship report submission. The journal does not need to be reviewed and endorsed by your workplace supervisor.

Note:
- Remove the rows that are not required.
- If you are absent due to In-Camp Training or other approved reasons, please indicate accordingly with the dates of your absence.

Week No.	Key Learning Point
Week 1	**Understanding RL locomotion and the B2W platform** I studied reinforcement learning and PPO and reviewed the Unitree B2W documentation, including its controller and SSH workflow. This established how a policy trained in simulation can later be integrated with the robot's software and hardware interfaces.
Week 2	**Setting up an end-to-end simulation training environment** I configured Isaac Sim and Isaac Lab and used robot_lab to train a basic walking checkpoint. I also supported a multi-robot autonomous-navigation task, which showed me the importance of reproducible environments and team collaboration when integrating several robotics subsystems.
Week 3	**Validating policies across simulators** I deployed the trained teacher policy in Unitree MuJoCo for Sim2Sim validation and documented the training workflow in the project repository. I learned that observations, joint mappings, control rates, and controller settings must remain consistent for a policy to transfer reliably.

Week No.	Key Learning Point
Week 4	**Transferring and documenting robotics software** I refined the rough-terrain teacher-policy documentation and also tested an existing DeepRobotics autonomous waypoint-navigation system on my laptop. This taught me to trace dependencies, and record assumptions clearly so that a robotics workflow can be reused by others.
Week 5	**Designing terrain-specific expert policies** I trained stairs-up and slope-up teacher policies using specialised terrain configurations and reward functions, then documented the Isaac Lab terrain setup. I learned that expert performance depends on matching rewards, observations, and terrain difficulty to the intended skill rather than relying on one generic configuration.
Week 6	**Multi-expert distillation and fair policy comparison** I implemented multi-expert student-teacher distillation and created a performance matrix comparing teacher and student completion rates. I learned that distillation must be evaluated by whether the student retains each expert's behaviour and remains stable across different terrain conditions, not only by its training loss.
Week 7	**Connecting learned actions to low-level robot control** I documented the student-teacher distillation pipeline and studied how unitree_rl_lab handles the Unitree B2W controller, including PID control. This clarified that policy outputs must be converted into stable joint commands using correct action scaling and well-tuned low-level control.
Week 8	**Reproducing a research baseline before extending it** I trained a walking policy based on the predecessor to the Parkour paper and continued investigating the B2W controller. Reproducing a known method showed me the value of checking observations, rewards, and controller assumptions before introducing more complex skills or architectural changes.
Week 9	**Observation design and reward interpretation** I documented the reward functions in rough_env_cfg.py, verified that base linear velocity and elevation-map observations were enabled, and retrained the walking policy. I learned that a missing or misconfigured observation can silently limit learning, while reward terms must be understood as a complete system.

Week No.	Key Learning Point
Week 10	**Building a deployable depth-based student policy** I refactored the distillation pipeline, replaced privileged elevation maps with front and rear depth observations, added CNN encoders, and trained the student from the walking expert. This showed me how perception, memory, and proprioception must be combined while preserving a deployment-compatible observation interface.
Week 11	**Conservative RL fine-tuning after distillation** I studied and implemented RL fine-tuning for the walking student and examined how PPO hyperparameters should change when starting from an already trained policy. I learned that fine-tuning requires controlled updates and a stable value estimate so that robustness improves without destroying useful distilled behaviour.
Week 12	**Evaluating and consolidating the complete pipeline** I finalised the evaluation workflow, compared the expert, distilled student, and fine-tuned student on challenging terrain, and consolidated the implementation and findings for handover. This reinforced the importance of consistent success criteria and reproducible evaluation scripts when deciding whether a model change provides a genuine improvement.